

Java Database Connectivity (JDBC) - Full Guide

1. What is JDBC?

JDBC (Java Database Connectivity) is a Java API that enables Java programs to interact with databases. It provides a standard set of interfaces for accessing relational databases from Java code.

- JDBC allows execution of SQL statements from Java code.
- It helps in managing connections, queries, updates, and result sets.

Example Use Case: Java applications accessing MySQL, Oracle, or PostgreSQL databases.

2. JDBC Architecture

JDBC follows a layered architecture:

Layers:

1. **Java Application**
2. **JDBC API (Application Layer)**
 - Interfaces like `Connection`, `Statement`, `PreparedStatement`, `ResultSet`, etc.
3. **JDBC Driver Manager**
 - Loads and manages JDBC drivers.
4. **JDBC Driver (Driver Layer)**
 - Converts Java calls to DB-specific protocol.
5. **Database**
 - Executes SQL queries and returns results.

3. JDBC Driver Types

There are **4 types of JDBC drivers**:

Driver Type	Name	Description
-------------	------	-------------

Type 1	JDBC-ODBC Bridge	Connects through ODBC driver (Deprecated)
Type 2	Native-API Driver	Uses native DB libraries (Platform-dependent)
Type 3	Network Protocol Driver	Communicates via middleware server
Type 4	Thin Driver	Pure Java driver, directly communicates with the DB (Preferred)

4. Connecting to a Database

Steps to connect to a database using JDBC:

- Load the JDBC driver class**
`Class.forName("com.mysql.cj.jdbc.Driver");`
- Establish a connection**
`Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/db", "user", "password");`
- Create a statement or prepared statement**
`Statement stmt = con.createStatement();`
- Execute a query**
`ResultSet rs = stmt.executeQuery("SELECT * FROM users");`
- Process the result set**
`while (rs.next()) { System.out.println(rs.getString("name")); }`
- Close connection**
`con.close();`

5. Statement vs PreparedStatement

Feature	Statement	PreparedStatement
SQL Injection Risk	High	Low (uses parameter binding)
Performance	Slower	Faster (precompiled SQL)
Usage	Static queries	Dynamic or repeated queries
Syntax	<code>stmt.executeQuery(sql)</code>	<code>pstmt.setInt(1, id); pstmt.executeQuery()</code>

6. ResultSet Usage

ResultSet stores the results of a SQL query. It's a cursor that points to the current row of the result.

Common Methods:

- `rs.next()` → Move to the next row.
- `rs.getString("column_name")`
- `rs.getInt(1)` → Access by index.
- `rs.close()` → Close the ResultSet.

Example:

```
java
CopyEdit
ResultSet rs = stmt.executeQuery("SELECT name FROM users");
while (rs.next()) {
    System.out.println(rs.getString("name"));
}
```

7. Why We Use `com.` in Package Names

The prefix `com.` is used in Java package names to avoid naming conflicts.

- **Convention:** Use your organization's reversed domain name (e.g., `com.google`, `org.apache`)
- **Example:** `com.mycompany.database` ensures your code doesn't conflict with external libraries.

8. SQL Injection Example

● Unsafe Code (Vulnerable):

```
java
CopyEdit
String user = "admin";
String pass = "123";
String sql = "SELECT * FROM users WHERE username='" + user + "' AND password='" + pass + "'";
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery(sql);
```

✓ Safe Code (Using PreparedStatement):

```
java
CopyEdit
String sql = "SELECT * FROM users WHERE username=? AND password=?";
PreparedStatement pstmt = con.prepareStatement(sql);
```

```
pstmt.setString(1, user);
pstmt.setString(2, pass);
ResultSet rs = pstmt.executeQuery();
```

Why it's safe?

The PreparedStatement handles escaping and parameter binding, thus mitigating injection attacks.

9. Code Samples

a) Using Statement

```
java
CopyEdit
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM users");
while (rs.next()) {
    System.out.println(rs.getString("username"));
}
```

b) Using PreparedStatement

```
java
CopyEdit
PreparedStatement pstmt = con.prepareStatement("SELECT * FROM users
WHERE id = ?");
pstmt.setInt(1, 101);
ResultSet rs = pstmt.executeQuery();
```

c) Using CallableStatement

```
java
CopyEdit
CallableStatement cstmt = con.prepareCall("{call getUser(?, ?)}");
cstmt.setInt(1, 101);
cstmt.registerOutParameter(2, Types.VARCHAR);
cstmt.execute();
String name = cstmt.getString(2);
```

10. Hands-on Exercises

Exercise 1. Connect to a MySQL Database

- Use JDBC to connect and print data from a `products` table.

Exercise 2. Insert Data Using PreparedStatement

- Insert a new user into a `users` table securely.

Exercise 3. Update Data Using Statement

- Update a user's email where `user_id = 101`.

Exercise 4. Demonstrate SQL Injection & Fix

- First, use a vulnerable `Statement`.
- Then, fix it using `PreparedStatement` and explain the difference.

Exercise 5: Batch Insert Using PreparedStatement

Objective: Insert multiple rows efficiently using batching.

Exercise 6: Search Records by Partial Match

Objective: Use `LIKE` clause with placeholders.

```
Sample:
PreparedStatement pstmt = con.prepareStatement("SELECT * FROM products
WHERE name LIKE ?");
pstmt.setString(1, "%phone%");
ResultSet rs = pstmt.executeQuery();
```

Exercise 7: Delete Records Based on a Condition

Objective: Safely delete entries older than a certain date.

Exercise 8: Update Multiple Columns

Objective: Update user email and phone number using ID.

✓ *Confirm update by querying the user afterwards.*

Exercise 9: Dynamic Range Query with PreparedStatement

Objective: Fetch products within a price range.

Sample :

```
PreparedStatement pstmt = con.prepareStatement(
    "SELECT * FROM products WHERE price BETWEEN ? AND ?"
);
pstmt.setDouble(1, 100.00);
```

```
pstmt.setDouble(2, 500.00);  
ResultSet rs = pstmt.executeQuery();
```

Try changing the range and verify different results.

Exercise 10: Insert with Null Handling

Objective: Insert data where some fields are optional (e.g., description).

Sample

```
PreparedStatement pstmt = con.prepareStatement(  
    "INSERT INTO items (name, description) VALUES (?, ?)"  
);  
pstmt.setString(1, "Notebook");  
pstmt.setNull(2, java.sql.Types.VARCHAR);  
pstmt.executeUpdate();
```

Learn to handle null values in parameters.

Exercise 12: PreparedStatement Reuse

Objective: Reuse the same prepared statement for multiple parameter sets.

Reusing a PreparedStatement is better for performance.

Add on:

Java 6 and later: Driver Auto-Loading

In newer JDBC drivers (JDBC 4.0+), **manual `Class.forName()` is not strictly required** if the driver JAR has a file:

```
bash  
CopyEdit  
META-INF/services/java.sql.Driver
```

That file auto-registers the driver using Java's **Service Provider mechanism**.

However, using `Class.forName()` is still a good habit for:

- Backward compatibility
- Explicit control

Other Similar Methods and Classes

1. `ClassLoader.loadClass("com.example.MyClass")`

- Also loads a class, but **doesn't initialize** it (i.e., doesn't run static blocks).
- Used for finer-grained class loading control.

2. `Class.forName("com.example.MyClass", true, classLoader)`

- The second parameter (`true`) controls whether the class is initialized.

3. `ClassName.class`

- Used at compile time to get a `Class` object.

Example:

```
Class<?> clazz = String.class;
```

When to Use `Class.forName()` in JDBC?

Use it if:

- You're working with older JDBC drivers (pre-JDBC 4.0)
- You want to explicitly control driver loading
- You're not sure if the auto-loading mechanism is reliable